Project

Design and Implementation of a Lightweight Email Client

Computer Network Course Practice

2026 Fall

1 Description

This project asks you to design and implement a **lightweight email client** using socket programming. The goal is not to reproduce a complete commercial mail application. Instead, the required part focuses on a small but complete set of functions that demonstrate how application-layer network protocols work in practice.

The client should support **one email account**, send plain-text messages using a basic subset of SMTP, receive messages using a basic subset of POP3, display a simple Inbox, and save downloaded messages locally. A command-line interface is fully acceptable; a graphical user interface is optional.

For the required version, students should use a **local or instructor-provided test mail server whenever possible**. Real public mail providers often require TLS, OAuth, or provider-specific authentication; these are outside the minimum requirements unless the instructor explicitly provides a compatible setup.

The aim is to deepen your understanding of TCP socket programming, application-layer protocols, protocol parsing, client-side state management, error handling, and basic software design. Advanced functions such as attachments, TLS, IMAP, multiple accounts, and background synchronization are treated as **bonus features**.

2 Basic Requirements

The following functions constitute the required part of the project. Completing these functions correctly should be sufficient for a solid project submission.

2.1 Account Configuration

- Configure one mail account and one test server.
- Store the server address, SMTP port, POP3 port, username, and other necessary settings in a configuration file.
- The program should report clear errors when the server is unreachable or authentication fails.

2.2 Send Plain-Text Email with SMTP

The client must be able to send a plain-text email to one or more recipients. At minimum, support the following SMTP commands:

- `HELO` or `EHLO`;
- `MAIL FROM`;
- `RCPT TO`;
- `DATA`;
- `QUIT`.

The client should allow the user to enter at least:

- recipient address;
- subject;
- plain-text message body.

A high-level library that completely hides SMTP should not be used for the required protocol implementation. Standard socket and string-processing libraries may be used.

2.3 Receive Email with POP3

The client must be able to connect to a POP3 server, authenticate, list available messages, and retrieve a selected message. At minimum, support:

- `USER` and `PASS`;
- `STAT` or `LIST`;
- `RETR`;
- `QUIT`.

The Inbox should display basic information such as message number, sender, subject, and date when available. The user should be able to select one message and view its plain-text content.

2.4 Simple Local Mailbox

Downloaded messages should be saved locally so that they can be viewed again after restarting the program. The storage format is left to the student. Acceptable choices include:

- text files organized by message number;
- JSON or another simple structured file format;
- SQLite or another lightweight database.

Only a minimal local cache is required. Complex synchronization between local and remote mailboxes is not required.

2.5 User Interface

A command-line interface is sufficient. A typical interaction may include:

```
1. Check Inbox
2. Read Message
3. Compose and Send Email
4. View Local Messages
5. Exit
```

A graphical user interface is welcome but should not replace the networking requirements.

2.6 Error Handling and Logging

The client should handle common failures without crashing, including:

- connection failure and timeout;
- invalid username/password;
- unexpected server response;
- interrupted message transfer;
- invalid user input.

The program should keep a simple log of important events, for example:

```
[INFO] Connected to SMTP server
[INFO] Message sent successfully
[INFO] Retrieved message 12
[ERROR] POP3 authentication failed
[ERROR] Connection timeout
```

Passwords must never be written into the log.

3 Suggested Program Structure

A simple modular design is recommended:

- **SMTP module:** connect, send commands, parse responses, send messages;
- **POP3 module:** authenticate, list messages, retrieve messages;
- **Mailbox module:** save and load local messages;
- **UI module:** command-line menu or graphical interface;
- **Logger:** record important operations and errors.

The minimum project does **not** require a complex thread pool, a database server, or a full mail synchronization engine. If a GUI is used, network operations should preferably run in a worker thread so that the interface does not freeze.

4 Communication Protocol Examples

A simplified SMTP interaction may look like:

```
S: 220 test.example.edu SMTP ready
C: EHLO student-client
```

```
S: 250 OK
C: MAIL FROM:<alice@example.edu>
S: 250 OK
C: RCPT TO:<bob@example.edu>
S: 250 OK
C: DATA
S: 354 End data with <CRLF>.<CRLF>
C: Subject: Hello
C:
C: This is my first email client.
C: .
S: 250 Message accepted
C: QUIT
```

A simplified POP3 interaction may look like:

```
S: +OK POP3 server ready
C: USER alice
S: +OK
C: PASS password
S: +OK authenticated
C: LIST
S: +OK 3 messages
C: RETR 1
S: +OK message follows
S: [message data]
C: QUIT
```

Students should pay attention to line endings, multi-line responses, and the fact that a single socket read is not guaranteed to contain one complete protocol response.

5 Testing Requirements

The required testing is intentionally modest. At minimum, demonstrate:

1. successful connection to the SMTP and POP3 servers;
2. successful sending of at least three plain-text messages;
3. successful retrieval and display of received messages;
4. correct local saving and reopening of downloaded messages;
5. graceful handling of at least three error cases, such as wrong password, unavailable server, or timeout.

Students should briefly measure and report the time needed to send and retrieve messages under normal conditions. A small data table is sufficient; a simple figure is optional. Extensive stress or load testing is not required for the basic project.

6 Bonus Features

After completing the basic requirements, students may implement additional features for bonus credit. The following list is intentionally open-ended; quality is more important than quantity.

- **TLS/SSL support:** protect SMTP/POP3 traffic using TLS or STARTTLS.

- **MIME attachments:** send and receive one or more file attachments.
- **Background checking:** periodically check for new mail in a worker thread.
- **IMAP support:** replace or supplement POP3 with a small subset of IMAP.
- **Multiple accounts:** switch between two or more configured accounts.
- **Search:** search local messages by sender, subject, or keyword.
- **Message management:** mark messages read/unread, delete messages, or organize them into local folders.
- **HTML email:** display a safe subset of HTML messages.
- **Graphical interface:** implement a clear and usable desktop GUI.
- **Improved concurrency:** allow sending, receiving, and local operations to proceed concurrently without race conditions.

Students may propose other technically meaningful extensions after discussing them with the instructor.

7 Suggested Report Structure

Use the course report template and organize the report around the following sections:

1. Requirements;
2. Analysis;
3. Design;
4. Development and Implementation;
5. System Deployment, Startup, and Use;
6. System Test;
7. Performance;
8. Summary or Conclusions;
9. References;
10. Comments (optional course comments or suggestions).

The report should focus on the implemented SMTP/POP3 protocol flow, TCP stream handling, program structure, normal and error-case tests, and the brief performance measurement described above. Cite the protocol documents, libraries, and other materials actually used; no fixed reference count is required unless the instructor announces one separately.

8 Deliverables

The final submission should contain:

- complete source code;
- a README describing compilation, configuration, and execution;

- one report following the course report template and explaining the architecture and the implemented SMTP/POP3 protocol flow;
- screenshots or terminal output demonstrating the main functions;
- test results and representative log files;
- a short description of any bonus features implemented.

All work should be completed independently. Students should understand every part of the submitted code and be prepared to explain the protocol behavior during the presentation or demonstration.